Как встроить часть участника 5 (Android + QA/Security)

Подготовил участник 3 (backend + DevOps), 31.10.2026. Для тимлида и его Claude Code: встраивать по той же схеме, что части участников 2, 3 и 4 (`docs/AGENT_SDK.md` §13, примеры — `agents/system_architect`, `agents/backend_coder`, `agents/devops`). Что уже изменено при встраивании участника 3 — `Changes.md` в корне.

1. Что есть у участника 5

Часть	Что внутри	Где было на vm5
Android-каркас	Kotlin 2.2, Compose, Material 3, Retrofit + kotlinx.serialization, FusedLocation, WorkManager; `AGENT_CONTRACT.md` (куда агенту можно писать), `frozen.sha256` (файлы ядра, менять нельзя)	`~/android-image/android`
Шлюз Android	`tools/android_gate/screens.py` — скриншот-тесты экранов из `03_ux/screen-spec.android.json`	`~/android-image/tools`
Образ сборки	`specforge/android:jdk21-api36` (JDK 21, Android SDK 36, прогретый Gradle-кэш; на vm5 Maven Central закрыт — собирать только в нём)	`~/android-image/infra/images/android`
Android-агент	экраны по UX-спеке, геолокация, таймер сессии, уведомления, unit-тесты	у участника 5
QA / Security	Schemathesis по OpenAPI, e2e демо-сценарий, Semgrep, Trivy, gitleaks, OWASP ZAP; отчёты JUnit, SARIF, `findings.json`, `report.md`	у участника 5
CI	`tests.gitlab-ci.yml`, `security.gitlab-ci.yml`	у участника 5

2. Пакет

1. Код — пакетом `packages/specforge-mobile/` (или два: `specforge-android`, `specforge-qa`) со своим `pyproject.toml` и тестами. Имя Python-пакета — `specforge_mobile` и т. п.: **не** `agents`, `tools`, `scripts` — они конфликтуют с одноимёнными пакетами платформы (так же сделано у участников 3 и 4).
2. Корневой `pyproject.toml`: пакет в `dependencies`, `[tool.uv.sources]` (`{ workspace = true }`), тесты — в `testpaths`, модуль — в mypy-override (по образцу `specforge-backend`).
3. Модель — только через `ctx.llm`. Если у части свой клиент — мост в `agents/_team/` по образцу `agents/_team/backend.py` (`BackendLLM`: протокол части поверх `LLMClient.chat`).
4. В `specforge/` и `agents/` — без слов предметной области кейса (`tests/domain_leak_words.txt`); в `packages/` можно.

3. Агент `android_coder` (этап `06_android`)

- `AgentSpec(stage="06_android", depends_on=["system_architect", "ux_designer"])`, входы: `02_system/openapi.yaml`, `03_ux/screen-spec.android.json`, `01_requirements/profile.json`.
- **Включать только если нужен мобильный клиент**: `01_requirements/profile.json → needs_mobile_app` (`AgentSpec.enabled_if`, см. как `agents/_team/web.py` выбирает платформы экранов).
- **Kotlin-клиент API генерирует часть участника 3**, без LLM — вызвать первым шагом агента:

  ```
  CLIENTS=kotlin ANDROID_APP_ID=<appId> PYTHON=<python платформы> \
    packages/specforge-backend/generators/gen-clients.sh
  <artifacts>/02_system/openapi.yaml <artifacts> --verify
  ```

  Результат — `06_android/app/src/main/kotlin/<appId>/core/network/generated` (пакет `<appId>.core.network.generated`; кейс 1 — `com.specforge.case1`), это совпадает с каркасом участника 5 (`namespace com.specforge.app` — ядро, `applicationId` — от кейса). Нужны JDK 21 и доступ к Maven Central: на vm5 его нет → запускать в образе `specforge/android:jdk21-api36` или убрать `--verify` (без компиляции клиента). Каталог `generated/**` агенту править нельзя (`protected_globs`).
- Каркас копировать в `06_android/` (выход этапа очищается оркестратором перед запуском).
- Шлюз — как у `frontend_coder`: `sandbox_image="specforge/android:jdk21-api36"`, `gate_commands=["./gradlew assembleDebug testDebugUnitTest"]`, `protected_globs` — из `frozen.sha256` и `core/**`; красный шлюз чинит ремонтник платформы. Скриншоты — в `06_android/screenshots/` (их читает техписатель и VLM-ревьюер участника 4).
- Публикация: `config/repo_layout.yaml → 06_android: android/` — уже есть.

4. Агент `qa` (этап `07_tests`)

- `AgentSpec(stage="07_tests", depends_on=["backend_coder", "devops"])` (+ `android_coder`, если нужен).
- **Стенд.** DevOps-агент поднимает стенд только на время своего шлюза и гасит его. QA поднимает сам:

  ```
  docker compose --project-directory <artifacts>/08_infra up -d --build --wait   #
  backend на :8080
  # ... Schemathesis / e2e / ZAP против http://127.0.0.1:8080 ...
  docker compose --project-directory <artifacts>/08_infra down
  ```

  Swagger: `GET /swagger/v1/swagger.json`, health: `/health/ready`, метрики: `/metrics`. Секреты стенда — в `08_infra/.env` (создаёт `gen-infra.sh`, в git не попадает). Compose-проект называется `specforge-<имя каталога над artifacts>`; другие стенды `specforge*` держат те же порты — останавливайте их (так делают `deploy.sh` и DevOps-агент).
- **Учётные записи для тестов:** операции входа и роли берутся из контракта (`x-roles`). У каждого кейса свои роли; роль платформы — `admin`. JWT подписывает backend ключом `JWT_SIGNING_KEY` из `.env`.
- **Формат ошибок** — RFC 9457 `application/problem+json`, поле `code`; ошибки валидации — словарь `errors: {"Points[0].RecordedAt": ["…"]}` (не массив).
- **События Kafka** — конверт `{eventId, type, version, occurredAt, source, partitionKey, payload}`, доставка через outbox, at-least-once. `TIME_SCALE` в `.env` ускоряет таймеры предметной области (e2e с таймерами).

- **Интеграционные тесты backend уже есть**: агент участника 3 пишет их по критериям приёмки (`04_backend/tests/App.IntegrationTests/Features`, Testcontainers) и матрицу `04_backend/traceability.md`; столбец `tests` в `02_system/traceability.csv` заполняет он же. QA дописывает туда свои ID (Schemathesis/e2e), не затирая чужие значения.
- Отчёты: `07_tests/` → JUnit XML, SARIF (Semgrep, gitleaks, Trivy), ZAP JSON + HTML, `security/findings.json`, `security/report.md`. Публикуются в `reports/tests/`, техписатель читает `07_tests/**`.

5. CI продукта

- Корневой `.gitlab-ci.yml` генерирует участник 3 (`08_infra/.gitlab-ci.yml`, публикатор кладёт его в корень). Стадии: `build → test → package → security → deploy → e2e`; package стоит **до** security, чтобы Trivy сканировал готовые образы `$API_IMAGE` (`specforge/backend:<sha>`) и `$WEB_IMAGE` (`specforge/web:<sha>`) без registry (shell-раннер на vm5).
- Файлы участника 5 подключаются через `include: local`, если существуют: `ci/tests.gitlab-ci.yml`, `ci/security.gitlab-ci.yml` **или** `reports/tests/ci/…` (если агент `qa` пишет их в `07_tests/ci/` — так они окажутся в опубликованном репозитории). Задачи используют стадии `test` и `security`.
- e2e-стадия запускает все `07_tests/e2e/*.py --base $STAND_URL` после деплоя (в опубликованном репозитории путь `reports/tests/e2e` — при переносе e2e в агент `qa` поправить строку в `packages/specforge-backend/templates/ci/gitlab-ci.yml`, задача `stand:smoke`).

6. Документация и проверка

1. `agents/tech_writer/agent.py`: добавить `android_coder` и `qa` в `depends_on` (входы `06_android/screenshots/*` и `07_tests/**/*` уже есть).
2. `tests/test_team_agents.py`: добавить заглушки новых агентов в сквозной прогон — по образцу `NoDotnetBackend`, `NoDockerDevOps` (в тестах нет Gradle, Docker и стенда).
3. Перед отправкой:

```
uv sync
uv run pytest -q                                    # было 501 passed до участника 5
uv run ruff check . && uv run ruff format --check .
uv run mypy specforge agents
uv run specforge doctor                             # у каждого агента нужна папка
prompts/ (system.md)
```

4. Записать изменения в `Changes.md` (раздел «участник 5»).

7. Подводные камни, на которые уже наступали

- vm5: 4 CPU, 3,8 ГБ RAM; стенд ~0,5 ГБ (лимиты памяти в compose). Android-эмулятор и стенд одновременно на vm5 не поместятся — скриншоты снимать Robolectric/Paparazzi-тестами (как в `screens.py`), без эмулятора.
- На vm5 закрыт Maven Central: Gradle — только с прогретым кэшем образа `specforge/android:jdk21-api36`.
- Хосту конвейера нужны dotnet SDK 10 и Docker (агенты участника 3), а для Android — Docker с этим образом.

- Ключ LLM — только `SPECFORGE_LLM_API_KEY` в окружении, лимит 30 000 токенов в минуту на ключ — агентов запускать последовательно.